Hardware-Based

USB Write Blocker

For

Digital Forensics

Guide: Gautam Borkar

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

1. Introduction

In digital forensics, maintaining the integrity of digital evidence is a fundamental requirement. Any unauthorized modification to evidence can compromise investigations and affect legal admissibility. When USB storage devices are connected to a computer, operating systems may automatically perform write operations such as updating metadata or creating system files.

To address this issue, write blockers are used. This project focuses on the design and implementation of a hardware-based USB write blocker using Arduino Uno and a USB Host Shield, aimed at preventing any write operations on a USB storage device while allowing safe read access for forensic analysis.

2. Problem Statement

When a USB storage device is connected to a forensic workstation:

- The operating system may automatically mount the device in read-write mode
- File system metadata may be altered
- Hidden system files may be created

These actions violate forensic principles. Although hardware write blockers are commercially available, they are often expensive and inaccessible for academic or learning purposes. Hence, there is a need to explore a low-cost educational hardware-based solution.

3. Objective of the Project

The objectives of this project are:

- To design a hardware-based USB write-blocking mechanism.
- To prevent any write operations on USB storage devices.
- To understand USB host-device communication.
- To gain practical experience with forensic hardware tools.
- To study real-world challenges in hardware-based forensic implementations.

Hardware Writer block

A hardware write blocker is a physical device placed between a storage medium and a computer to ensure that no write commands reach the storage device.

Key Features:

- Physically blocks write commands
- Independent of operating system behavior
- Widely used in professional forensic investigations

This project attempts to implement a prototype-level hardware write blocker for educational purposes.

5. Hardware Components Used

- Arduino Uno R3
- USB Host Shield
- USB Male and Female connectors
- Breadboard
- LEDs and resistors
- Jumper wires

6. Working Principle of the Hardware Write Blocker

The hardware write blocker is designed to work as follows:

1. The USB storage device is connected through the USB Host Shield
2. Arduino acts as an intermediary between the computer and the USB device
3. USB communication is monitored using host shield libraries
4. Write commands are intended to be blocked or ignored
5. Visual indicators (LEDs) are used to show device status

7. Implementation and Experimental Work

- Initial hardware setup and component testing were completed successfully
- LED and resistor testing confirmed correct power and signal flow
- USB Host Shield libraries were studied and configured
- USB device detection was partially achieved

However, during advanced stages of implementation, the USB Host Shield did not function reliably and failed to consistently detect or manage USB storage devices as expected.

8. Limitations Encountered

- USB Host Shield showed inconsistent behavior
- USB storage devices were not always recognized correctly
- Write-blocking logic could not be fully implemented due to hardware instability
- Limited debugging support for low-level USB operations

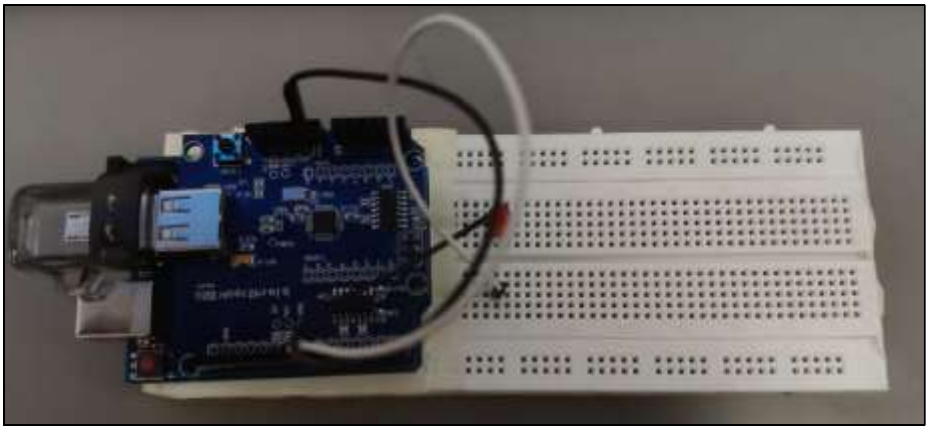
Code:

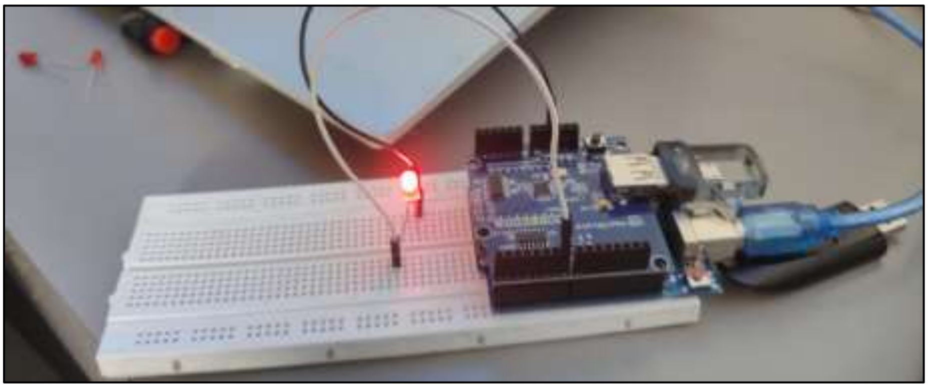
```
#include <Usb.h>

#include <usbhub.h>

#include <masstorage.h>

#define LED_PIN 7

USB     Usb;
```

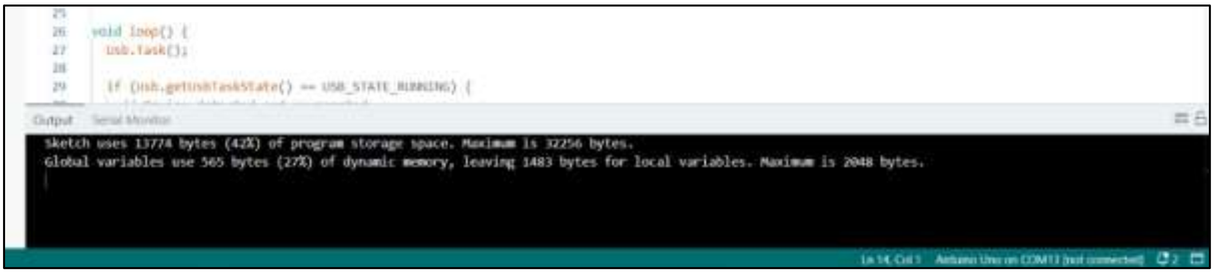
```
USBHub  Hub(&Usb);
BulkOnly Bulk(&Usb);
void setup() {
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);
  Serial.begin(9600);
  delay(2000);
  if (Usb.Init() == -1) {
    Serial.println("USB Host Shield Init Failed");
    while (1);
  }
  Serial.println("USB Host Shield Ready");
}
void loop() {
  Usb.Task();
  if (Usb.getUsbTaskState() == USB_STATE_RUNNING) {
    digitalWrite(LED_PIN, HIGH);
    delay(300);
    digitalWrite(LED_PIN, LOW);
    delay(300);
    Serial.println("Pendrive DETECTED");
  } else {
    digitalWrite(LED_PIN, LOW)}}
```

Output:


USB Host Shield Library 2.0
REMOVE






void loop() {
usb.Task();
if (usb.getUsbTaskState() == USB_STATE_RUNNING) {
Output
Serial Monitor
Sketch uses 13774 bytes (42%) of program storage space. Maximum is 32256 bytes.
Global variables use 565 bytes (27%) of dynamic memory, leaving 1483 bytes for local variables. Maximum is 2048 bytes.

9. Conclusion

This hardware-based USB write blocker project was almost completed**,** but due to improper and inconsistent functioning of the USB Host Shield, the final write-blocking mechanism could not be fully implemented. Despite this limitation, the project provided valuable hands-on experience in hardware interfacing, USB communication, and forensic tool design.

Through this project, we learned how hardware write blockers function, how USB host-device communication works, and the practical challenges involved in implementing forensic hardware solutions. The project enhanced our understanding of evidence protection, hardware troubleshooting, and real-world constraints faced during forensic tool development. Although the system could not be completed entirely, the learning outcomes achieved make this project successful from an academic and practical perspective.